AI-SHIELD

Rule-Based Content Moderation Layer
Home Assignment

Issued Date:	August 14, 2026
Candidate:	Mohit Kumar
Role:	EM QE

Assignment Overview:
Complete the development and integration of a rule-based content moderation layer that inspects, sanitizes, and gates inbound text/file payloads before they reach a downstream LLM. This is a comprehensive full-stack assignment covering backend implementation, frontend UI, integration testing, and API documentation.

Table of Contents

Assignment 1: Backend Implementation

Objective:

Implement a complete Node.js/Express backend service that orchestrates multiple detector APIs and applies rule-based decisions on incoming payloads. The system must handle both synchronous text analysis and asynchronous file processing.

Key Requirements:

✓ Orchestrator Service: Sequence detectors (PII, CII, Secrets, Prompt Injection) with precedence

✓ Decision Engine: Apply BLOCK / MASK / WARN / ALLOW rules deterministically

✓ Async File Queue: Implement BullMQ/Redis for large file processing (>5MB)

✓ API Endpoints: Health check, moderate (text/file), detector routes, rules management

✓ Rule Packs: JSON-based, hot-reloadable rule configurations per detector

✓ Audit Logging: Track all decisions with requestId, processing time, findings

✓ Error Handling: Graceful degradation for detector failures

✓ Jest Unit Tests: Minimum 80% code coverage for detectors and orchestrator

Repository:

https://github.com/cyclone0077/ai-shield.git

Detectors to Implement/Complete:

PII Detector	Email, phone, SSN, Aadhaar, PAN, bank account patterns
CII Detector	API keys, tokens, database connections, config secrets
Secrets Detector	AWS/GCP/Azure credentials, private keys, OAuth tokens
Prompt Injection	Role-play attempts, instruction overrides, jailbreak patterns
File Validator	Extension + MIME + magic-byte agreement, text extraction
Token Budget	Count tokens; reject if exceeds configured limit

Deliverables:

• Complete, tested detector implementations (PII, CII, Secrets, Prompt Injection, File Validator, Token Budget)

• Orchestrator with decision precedence logic and hot-reload capability

• BullMQ-based async queue for large files with job status polling

• Full API test suite (Jest + Supertest) with golden test sets per detector

• Environment configuration (`.env.example` with all required variables)

• README with setup, testing, and API documentation

Assignment 2: Frontend Implementation

Objective:

Build a React + Vite + TypeScript frontend that allows users to test the moderation layer interactively. Display detailed findings, sanitized content, and decisions with a polished, accessible UI.

Key Requirements:

✓ Text Input Form: Paste or type content for real-time analysis

✓ File Upload: Drag-and-drop or file picker for document testing

✓ Decision Display: Visual indicators for BLOCK (red), MASK (yellow), WARN (orange), ALLOW (green)

✓ Findings Panel: Detailed view of detected issues (stage, type, ruleId, severity)

✓ Sanitized Output: Show masked/redacted content side-by-side with original

✓ Token Counter: Display token count vs. budget with visual progress bar

✓ Job Status Polling: Track async file jobs in real-time

✓ Rules Management UI: View and hot-reload active rule packs (admin only)

✓ Responsive Design: Mobile-friendly, accessibility (WCAG 2.1 AA)

✓ Vitest + Testing Library: Minimum 70% component coverage

Repository:

https://github.com/cyclone0077/ai-shield.git

UI Components to Build:

Input Panel	Text textarea + file upload with drag-and-drop, loading state
Decision Badge	Large visual indicator (BLOCK/MASK/WARN/ALLOW) with icon
Findings Table	Sortable/filterable list of detected issues per detector
Sanitized View	Side-by-side comparison: original vs. redacted content
Token Gauge	Progress bar showing token count and budget headroom
Job Tracker	Polling widget for async file jobs (status, ETA)
Rules Panel	List of active rule packs with reload button (protected)

Deliverables:

• React components for all UI elements (TypeScript, composable, reusable)

• Vite-based build with dev server and production optimization

• Complete styling (CSS modules or Tailwind; ensure dark mode support)

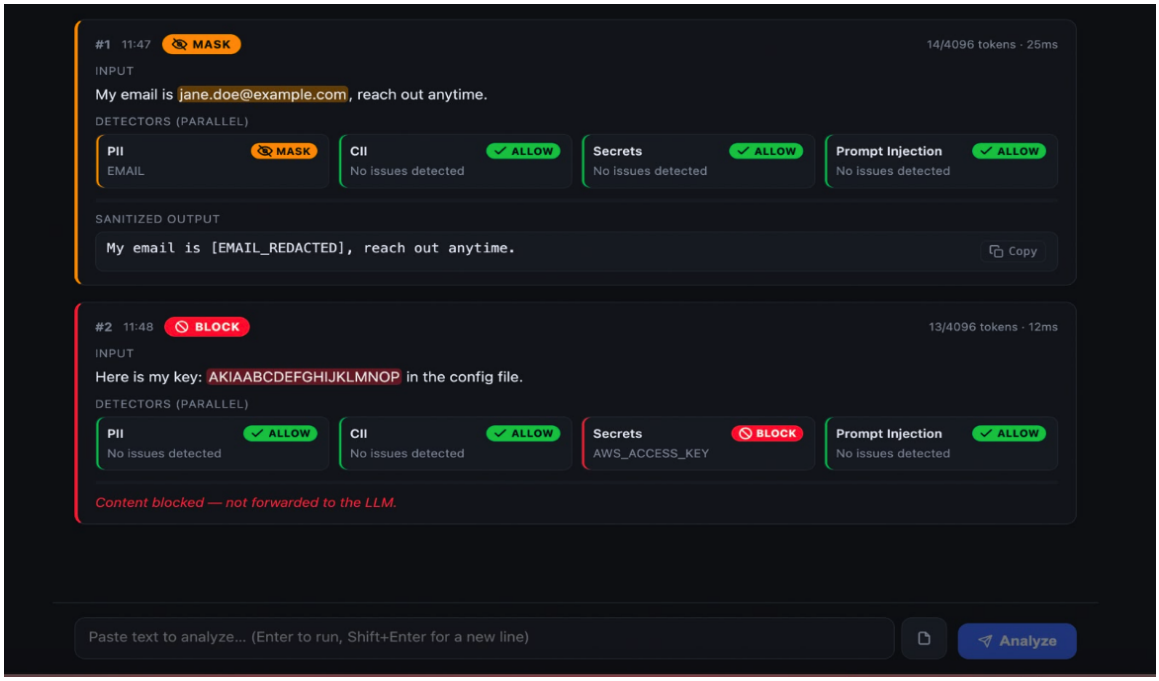
• API client wrapper with error handling and retry logic

• Vitest + React Testing Library unit/integration tests

• Accessibility audit (keyboard navigation, ARIA labels, color contrast)

• Responsive design tested on mobile (375px) and desktop (1920px)

UI Example - Moderation Interface:



Assignment 3: Integration - TestCase Generator + Moderation Layer

Objective:

Integrate the moderation layer with the existing TestLeaf test case generator. This ensures that AI-generated test cases are scanned for sensitive data (PII, secrets) before storage or display, and that compliance/KYC test cases don't accidentally leak customer data.

Key Requirements:

✓ TestLeaf Integration: Pre-moderate AI-generated test cases before saving to MongoDB

✓ Compliance Layer: Flag test cases that reference real customer data or credentials

✓ Sanitization Pipeline: Auto-mask PII in test case descriptions and assertions

✓ Audit Trail: Log all moderation decisions tied to test case generation events

✓ Graceful Fallback: If moderation is slow, queue test case generation (don't block)

✓ Dashboard Integration: Show moderation status in TestLeaf UI (pass/fail/warn)

✓ E2E Test Suite: Verify AI-generated → moderated → stored flow end-to-end

Repository:

https://github.com/cyclone0077/langchain-multimodal-aishield.git

Integration Points:

TestLeaf	Call moderation API after test case generation, before save
MongoDB	Store moderation metadata in test case document
Dashboard	New 'Moderation Status' column in test case list view
CI/CD	Add moderation check to test suite pipeline (fail if BLOCK)
Alerts	Notify admins if PII is detected in generated test cases

Deliverables:

• Middleware/wrapper that intercepts test case generation and calls moderation API

• Enhanced test case document schema to include moderation metadata

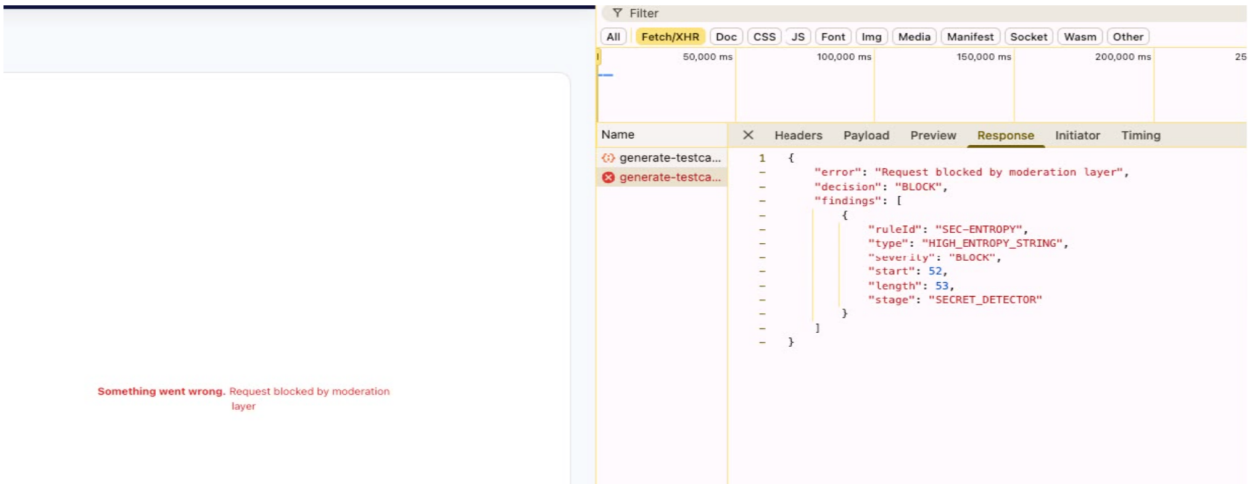
• Dashboard UI extension showing moderation status per test case

• E2E test suite covering happy path (ALLOW), rejection (BLOCK), masking (MASK)

• Documentation on moderation-aware test case generation workflow

• Performance benchmarks (moderation latency vs. test generation throughput)

Integration Example - Browser DevTools:


Something went wrong. Request blocked by moderation layer
Filter
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
50,000 ms
100,000 ms
150,000 ms
200,000 ms
Name
generate-testca...
generate-testca...
Headers
Payload
Preview
Response
Initiator
Timing
{
"error": "Request blocked by moderation layer",
"decision": "BLOCK",
"findings": [
{
"ruleId": "SEC-ENTROPY",
"type": "HIGH_ENTROPY_STRING",
"severity": "BLOCK",
"start": 52,
"length": 53,
"stage": "SECRET_DETECTOR"
}
]
}

Assignment 4: Postman Collection & API Documentation

Objective:

Create a comprehensive Postman collection documenting all moderation layer API endpoints. Include request/response examples, error scenarios, and pre/post-request scripts for integration testing.

API Endpoints to Document:

GET	/health	Liveness check
POST	/api/v1/moderate	Orchestrator: full pipeline on text/file
POST	/api/v1/file/validate	File validator (ext + MIME + magic bytes)
POST	/api/v1/detect/pii	PII detector (email, phone, SSN, Aadhaar, PAN)
POST	/api/v1/detect/cii	CII detector (API keys, tokens, DB strings)
POST	/api/v1/detect/secrets	Secrets detector (AWS/GCP credentials)
POST	/api/v1/detect/prompt-injection	Prompt injection detector
POST	/api/v1/validate/tokens	Token budget validator
GET	/api/v1/rules	List active rule packs
POST	/api/v1/rules/reload	Hot-reload rule packs (admin only)
GET	/api/v1/jobs/:id	Poll async file job status

Postman Collection Requirements:

✓ Complete folder structure by detector and workflow

✓ Request/response examples for each endpoint (success + error cases)

✓ Environment variables (BASE_URL, ADMIN_API_KEY, file upload paths)

✓ Pre-request scripts: Generate mock payloads, set timestamps

✓ Post-request scripts: Extract jobId for async tracking, assert response schema

✓ Test cases: Validate HTTP status, response schema, decision correctness

✓ Golden test sets: Pre-defined payloads covering all detector scenarios

✓ Performance tests: Measure latency per detector, identify bottlenecks

Test Scenarios to Cover:

Happy Path (ALLOW)	Clean text with no issues detected
PII Detection	Email, phone, Aadhaar, PAN, SSN in text
CII Detection	API keys, database URLs, config strings
Secrets Detection	AWS/GCP/Azure credentials, private keys
Prompt Injection	Role-play, jailbreak, instruction override attempts
Mixed Findings	Multiple detectors trigger in same request
Large File Upload	Async queue job, polling status

Token Overflow	Payload exceeds token budget, rejected
Malformed Request	Missing required fields, invalid JSON
Rate Limiting	Exceed request rate, 429 response

Deliverables:

• Postman collection (JSON export) with all endpoints and examples

• Environment configuration file with placeholders for local/staging/prod

• Request body templates and response schema validations

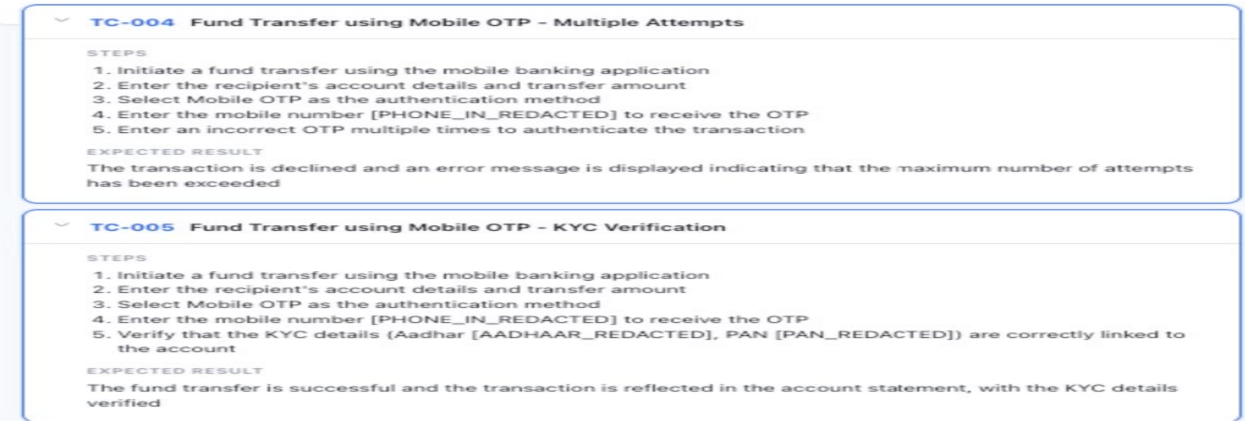
• Test scripts (pre-request + post-request) for each endpoint

• Golden test set (JSON payloads covering all detector scenarios)

• Performance baseline (latency per detector, throughput benchmarks)

• Postman documentation (markdown export or collection-level descriptions)

Test Case Reference - Fund Transfer with OTP Validation:



Project Architecture & Stack

Backend Stack:

Component	Technology
Runtime	Node.js 18+
Framework	Express.js
Testing	Jest + Supertest
Async Queue	BullMQ + Redis
Logging	Winston or Pino
Validation	Joi or Zod

Frontend Stack:

Component	Technology
Framework	React 18+
Build Tool	Vite
Language	TypeScript
Testing	Vitest + React Testing Library
Styling	Tailwind CSS or CSS Modules
State	React Hooks (Context/Reducer)

Project Structure:

```
ai-shield/
  src/
    detectors/ → pii, cii, secret, prompt-injection, file-validator, token-validator
    orchestrator/ → sequencing + decision engine
    queue/ → BullMQ producer/consumer
    routes/ → health, rules, jobs
    shared/ → entropy, luhnCheck, masker, ruleMatcher, auditLogger
  config/rules/ → JSON rule packs (PII, CII, secrets, prompt injection)
  frontend/ → React + Vite + TypeScript
  tests/ → Jest unit/integration tests
```

Setup Instructions

Prerequisites:

• Node.js 18+ and npm installed
• Redis server running (for async queue)
• Git installed
• Postman installed (for API testing)

Backend Setup:

1. Clone and install:

```
git clone https://github.com/cyclone0077/ai-shield.git
cd ai-shield
npm install
```

2. Configure environment:

```
cp .env.example .env
# Edit .env with your Redis URL, port, API keys
```

3. Start Redis (if not running):

```
redis-server
```

4. Run backend:

```
npm run dev # API server
npm run worker # BullMQ worker (separate terminal)
```

Frontend Setup:

```
cd frontend
npm install
npm run dev # Vite dev server on http://localhost:5173
```

Testing:

```
# Backend tests
npm test
npm run test:coverage

# Frontend tests
npm run frontend:test
npm run frontend:test:ui
```

Submission Guidelines

Submission Checklist:

✓ Backend code committed to GitHub with clear commit messages

✓ Frontend code committed to same/separate repo with component documentation

✓ All tests passing (minimum 80% backend, 70% frontend coverage)

✓ README.md with setup, configuration, and API overview

✓ Environment file template (.env.example) with all required variables

✓ Postman collection exported as JSON, with all endpoints documented

✓ Integration tests demonstrating end-to-end moderation flow

✓ Performance benchmarks (latency per detector, throughput)

✓ Accessibility audit results (WCAG 2.1 AA compliance)

✓ Documentation of any known limitations or deferred features

Deliverables Summary:

Backend (Node.js/Express)	src/ folder with all detectors, orchestrator, queue, tests
Frontend (React/Vite)	frontend/ folder with components, tests, build output
Integration	langchain-multimodal-aishield repo with TestLeaf integration
API Documentation	Postman collection (JSON) + README API reference
Configuration	.env.example with all required variables
Testing	Jest (backend), Vitest (frontend), golden test sets

Evaluation Criteria

Scoring Breakdown:

Criterion	Weight	Description
Backend Implementation	30%	Detector quality, orchestrator logic, error handling, test coverage
Frontend Implementation	20%	UI/UX, responsiveness, accessibility, component quality
Integration	15%	TestLeaf moderation pipeline, end-to-end testing, documentation
API Documentation	15%	Postman collection completeness, test scenarios, examples
Code Quality	10%	Readability, maintainability, adherence to best practices
Performance	5%	Latency benchmarks, async queue efficiency, throughput
Documentation	5%	README, inline comments, setup clarity, architecture docs

Quality Standards:

- **Code Coverage:** Backend ≥ 80%, Frontend ≥ 70%
- **Linting:** ESLint + Prettier (zero warnings)
- **Type Safety:** TypeScript strict mode, no 'any' types
- **Testing:** Unit tests + integration tests for all critical paths
- **Documentation:** JSDoc for all public functions, README setup instructions
- **Performance:** API response time < 500ms (sync), latency benchmarks provided
- **Accessibility:** WCAG 2.1 AA minimum (tested with axe or lighthouse)
- **Security:** No hardcoded secrets, environment variables for all config

Important Notes

Dependencies: Redis is mandatory for async file queue. Install and run before starting backend.

Rule Packs: Keep rule packs (JSON) in config/rules/. Use hot-reload endpoint to test changes without restart.

Async Jobs: For large files, always poll /api/v1/jobs/:jobId. Don't assume synchronous response.

Error Handling: If any detector fails (e.g., external regex error), orchestrator must gracefully degrade. Log it.

Git Workflow: Use feature branches. Commit frequently with clear messages. Final submission must be on main.

Communication: If stuck on a requirement, document the blocker and propose a workaround. Don't leave it unfinished.